

COMS TEST DAY GUIDE

6 October 2026 | 14:30-17:00 | Lab computer | Qoder allowed | Other AI models prohibited

THE GOAL

Finish with a **working product**, strong **rubric coverage**, a clear **README**, and a **public repository pushed before 17:00**. Do not sacrifice a working submission for unnecessary polish.

Suggested timing

14:30-14:40	Read brief + rubric, create/clone repo, open in Qoder
14:40-15:50	Build the minimum working product and high-value requirements
15:50-16:15	Complete remaining important rubric items
16:15-16:35	Rubric audit, debugging and testing
16:35-17:00	Stabilise, README, final test, commit, push and verify

1. WHEN THE TEST OPENS

- **Read the whole brief yourself first.**
- **Read the entire marking rubric.** The rubric tells you where the marks actually are.
- Check for required technologies, naming rules, file structure, endpoints, features, tests or other constraints.
- Do not immediately tell Qoder to build everything.

Create the submission repository

On GitHub: **New repository** -> **required/sensible name** -> **Public** -> **Add README** -> **Create repository**.

Public is critical: the marker must be able to clone the repository without an SSH key or HTTP PAT.

Clone it onto the lab computer using HTTPS:

```
git clone https://github.com/YOUR-USERNAME/YOUR-REPO.git
cd YOUR-REPO
```

Open this exact cloned folder in Qoder. Everything Qoder creates should live inside the Git repository.

2. FIRST QODER INTERACTION

Copy the **complete assessment brief and complete rubric** from Moodle. Then use the master prompt on the next pages.

Do not blindly accept Qoder's plan. Read it and confirm that it has understood the task and the marking priorities.

Once the plan looks correct, tell it:

```
Start implementing the minimum working version now. Prioritise the highest-value rubric requirements first. Work directly in this repository. Keep the solution simple and get a working version running as early as possible.
```

MASTER QODER PROMPT

Paste the full brief and rubric into the indicated sections.

You are helping me complete a 2.5-hour university coding assessment.

I will provide the complete assessment brief and marking rubric below.

Your goal is to help me produce the highest-scoring **WORKING** submission possible within the time limit.

IMPORTANT RULES:

1. Read the ENTIRE brief and rubric before writing code.
2. First, analyse the requirements and give me:
 - A short checklist of everything required.
 - The marks/importance of each requirement based on the rubric.
 - The order we should implement them in.
 - Any ambiguous requirements or potential traps you notice.
3. Prioritise:
 - A working application.
 - High-mark rubric items.
 - Required functionality.
 - Correctness.
 - Easy-to-run setup.before visual polish or optional features.
4. Work directly in the current repository.
5. Before implementing something, inspect the existing project structure and code. Do not unnecessarily replace working code.
6. Build incrementally. Get a minimal working version running as early as possible, then add features.
7. Keep the project simple. Do not introduce unnecessary libraries, frameworks, abstractions, architecture or complexity unless they directly help satisfy the brief.
8. After each major implementation stage:
 - Check for obvious errors.
 - Make sure the project still builds/runs.
 - Tell me briefly what has been completed.
 - Tell me what requirement we should tackle next.
9. If you encounter an error, investigate the actual cause rather than repeatedly making speculative changes.
10. Do not invent requirements that aren't in the assessment.
11. Pay close attention to exact wording in the rubric. Implement functionality in a way that clearly demonstrates each rubric criterion to the marker.
12. Keep track of the rubric throughout the assessment. Maintain a checklist of:
 - COMPLETE
 - PARTIAL
 - NOT STARTED
13. Do not spend excessive time polishing UI unless UI/design carries significant marks.
14. Make sensible decisions yourself when something is minor or obvious instead of constantly asking me questions. Ask me when a decision could materially affect the assessment requirements.
15. Make sure errors and edge cases are handled where appropriate.

16. Before we finish, perform a FINAL RUBRIC AUDIT:
- Go through every rubric item individually.
 - Identify anything missing or partially implemented.
 - Fix high-value missing items first.
 - Check that the application actually runs.

17. Create/update README.md with EXACT instructions a marker needs to run the project from a fresh clone.

The README must include things such as:

- Prerequisites if required
- Installation commands
- Environment setup if required
- Exact command to start/run the project
- Any important usage instructions

The marker must be able to clone the repository and understand immediately how to run it.

18. Do not put secrets, passwords, API keys, tokens or credentials into the repository.

19. Near the end, remind me to:

- test the application myself
- check git status
- commit all required files
- push to GitHub
- verify the repository is public
- verify the final commit is visible remotely

Do NOT start coding until you have read and analysed the entire assessment brief and rubric.

Here is the assessment:

```
=====
PASTE COMPLETE BRIEF HERE
=====
```

Here is the marking rubric:

```
=====
PASTE COMPLETE RUBRIC HERE
=====
```

BUILDING & CHECKPOINTS

3. WHILE QODER IS BUILDING

- Run the application yourself as soon as there is a runnable version.
- If something breaks, give Qoder the **actual error output**. Do not paraphrase it if you can copy it.
- Keep checking the rubric. A feature worth many marks matters more than cosmetic polish.
- Avoid unnecessary rewrites of working code.
- Commit useful milestones so you always have a recoverable working state.

```
git status
git add .
git commit -m "Implement core functionality"
git push
```

4. RUBRIC AUDIT - ABOUT 45-60 MINUTES LEFT

Give Qoder this:

```
Stop adding optional features.

Perform a complete rubric audit now.

Go through every marking criterion one by one and compare it against the current implementation.

For each criterion, classify it as:
- COMPLETE
- PARTIAL
- MISSING

Estimate which missing or partial items are worth the most marks relative to the time required to fix them.

Then implement the highest-value fixes first.

Do not refactor working code unless necessary.
```

5. FINAL 20-30 MINUTES

Stop chasing optional features. From here, protect the submission and think like the marker.

Final Qoder prompt:

```
We are approaching the submission deadline.

Do a final submission check.

1. Make sure the application builds and runs.
2. Look for obvious runtime errors.
3. Check every rubric requirement.
4. Fix only important issues - no unnecessary refactoring.
5. Make sure README.md contains exact instructions for running the project from a fresh clone.
6. Check that all required files are present.
7. Check that no secrets or credentials are committed.
8. Tell me the exact commands I should run to verify the project and then commit and push it.

Do not make cosmetic changes unless they fix a rubric requirement.
```

6. README CHECK

The lecturer specifically requires a README with run instructions. The marker should not have to guess.

- State prerequisites if there are any.
- Give exact installation commands.
- Explain environment variables/setup if genuinely required.
- Give the exact start command.
- Mention any important usage steps needed to demonstrate the product.

Example only:

```
npm install  
npm run dev
```

FINAL SUBMISSION

7. FINAL GIT COMMANDS

Do this with enough time to recover from an authentication/network problem:

```
git status
git add .
git commit -m "Final submission"
git push
```

Then open the GitHub repository in the browser and verify:

- **Repository is PUBLIC.**
- **Latest commit is visible remotely.**
- **README.md is visible.**
- All important source files are present.
- No secrets, passwords, PATs or API keys were committed.
- You submit the correct repository URL.

The marker will use the last commit before 17:00. A change sitting only on the lab PC does not count. COMMIT + PUSH.

8. MARKER'S PERSPECTIVE

Before you finish, ask yourself:

- Can a stranger clone this repository without credentials?
- Can they immediately see how to run it?
- Do the README commands actually work?
- Does the application start?
- Can the marker clearly see the features the rubric awards marks for?
- Is my latest work actually on GitHub?

9. IF SOMETHING GOES WRONG

- **Qoder breaks working code:** use Git/history and return to the last working state rather than panicking.
- **Build error:** copy the exact error to Qoder and ask it to diagnose the root cause.
- **Running out of time:** abandon optional polish; fix high-mark missing items, README, test, push.
- **Git push issue:** address it early. Do not discover authentication problems at 16:59.
- **Feature isn't perfect:** a simple working implementation can be worth more than an ambitious broken one.

FINAL MENTAL CHECK

WORKING PRODUCT -> RUBRIC -> TEST -> README -> COMMIT -> PUSH -> VERIFY -> SUBMIT

Use only the tools permitted by the assessment rules. This guide is preparation material; during the assessment, follow any additional instructions given by the lecturer or tutors.